

Why the Optimizer Is Not Enough

Motivation for an LLM-supervised, human-gated Bayesian optimization framework

Y. Oksuzian

Mu2e — Phase-I Genesis proposal

2026-08-03

The thesis

Bayesian optimization solves the **inner** problem — find the best point in a given box, under a given objective, from a given simulation. Every campaign failure we have *measured* lived in the **outer** problem.

BO takes four things as **given**, and can validate none of them:

- 1 the **parameterization** — which coordinates describe the design;
- 2 the **bounds** — the box those coordinates live in;
- 3 the **objective and its noise** — what is being measured, how well;
- 4 the **fidelity of the simulation** producing every observation.

Across four production campaigns on the Mu2e stopping target, **each of these was wrong at least once**. None was detectable from inside the optimizer:

from within BO, a mis-specified problem looks exactly like a converged one.

Every number in this deck is measured, and traceable to a leaderboard row, a harvest `summary.json`, or a root-caused incident record.

Case study: the search space was wrong — twice

(a) Wrong coordinates. The 6D line optimized *extra foils on a pinned 37-foil base*. Re-parameterizing the *whole* target as $K=3$ quadratic profiles (radius, thickness, hole) + extent — a 10D redesign no optimizer proposes — found a strictly better design:

line	evaluations	$S/\sqrt{B}$	flash [MeV/POT]
6D extras-on-base	414	4.10	1.04×10^{-6}
10D profiles	80	4.41	9.2×10^{-7}

Both re-measured on the same Run1Bap stack via dedicated A/B arms — the comparison is apples-to-apples, not a scale artifact.

(b) Wrong bounds. The 10D box capped extent at 1100 mm. BO converged *onto the bound* — its two best front points sat exactly on it. A pure run reports **4.09** and stops: the correct answer to the wrong question.

Detecting it required acting *outside the parameter space*: read VirtualDetectorMaker.cc, classify each blocking volume as massless bookkeeping vs. real material, run worst-corner preflight probes (1400, 1700, 1800, 1900, 2000, 2100), then relocate a 20 μm *vacuum* disc by one config line.

⇒ **4.41 at** −12% **flash**, strictly dominating. Gain splits +0.32 from the bound, +0.34 from the search it unlocked.

The same pattern, four times — with measured cost

BO assumes	How it failed here	Cost
the box is right	extent capped at 1100 mm; BO piled onto the bound	champion 4.09 vs. 4.41 reachable, at higher flash
the objective is right	early flash metric was a <i>per-event mean</i> — blind to rate	two campaigns of NULL results ; later overturned: foils are a $2.5\times$ lever ($R^2=0.89$)
the simulation is faithful	grid tarball lacked the <code>holeRadii</code> patch, so jobs silently built <i>uniform-hole</i> stacks; preflight passed because the local env $\neq$ the grid tarball	all 62 rows of a line tainted and discarded
the noise model is right	GP left noise a free hyperparameter and fit $\sigma(S/\sqrt{B})=0.0507$ vs. a replicate-measured 0.0051 ($12\times$)	best-ever evaluation ranked 16th of 324 ; the picker optimized a surface with its own optimum erased

In all four cases **the optimizer was working perfectly**. The defect was upstream of it and invisible to it — a faster optimizer on a mis-specified problem is only a faster route to a confident wrong answer.

Each row is a root-caused, written-up incident, not an impression. Detection in every case needed something outside the loop: reading Offline source, cross-checking the grid environment against the local one, or knowing the experiment's replicate structure.

Optimization $\neq$ estimation — and the answer $\neq$ the decision

BO is a deliberately bad estimator. It does not spend evaluations where it is confident there is no improvement — correct for optimizing, disqualifying for measuring.

Live example. Does $S/\sqrt{B}$ keep rising with target length, or flatten? This decides whether we specify a **1.4 m** or a **2.0 m** stopping target.

route	evaluations	grid jobs	confounded?
keep running BO	~ 194 more	$\sim 25,000$	yes — shape co-varies
designed scan (fixed shape)	10	1,300	no

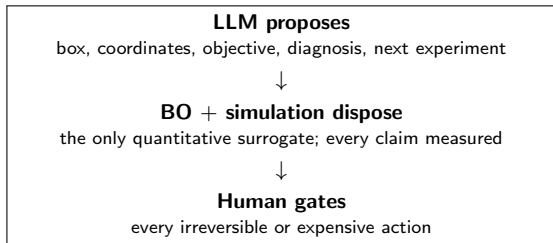
~ 194 is what the curvature term needs to reach even $t=2$; BO concentrates rather than spreads, so it is an optimistic bound. 130 grid jobs per evaluation.

And the reported optimum is not the engineering decision. The peak sits at 1770 vs. 2000 mm — a 0.017 difference in $S/\sqrt{B}$, below measurement noise and irrelevant to hardware. *Where the curve flattens* is what matters, and BO never asks. Likewise the most *deployable* point is not the champion: a different row gives +25% $S/\sqrt{B}$ at the deployed target's own flash.

$\Rightarrow$ Recognizing when to **stop optimizing and start measuring** is an outer-loop judgement — and worth $\sim 19\times$ the compute here.

What we propose — and how we design against its failure modes

The loop



Mu2e-aware means grounded in the actual stack: Offline source, geom files, Musings, FCL, and a persistent project wiki. That grounding is what identified EMC.Source as *massless* — and therefore movable — rather than a physical wall.

The claim is **not** that an LLM searches better than qNEHVI — it does not. It is that the optimizer's assumptions need an agent that can read the source, question the box, spot a confound, and know which question is being asked.

Honest failure modes, and the mitigation

risk	mitigation
fluent over-generalization (a wrong "constant volume" claim reached a slide)	every number traced to a measured source; human review gate caught it
plausible-but-wrong mechanism (a proposed gyration-phase story)	mechanisms are hypotheses to test , and this one was refuted by the data
no calibrated uncertainty	the GP remains the sole quantitative surrogate; the LLM never replaces the posterior
contaminating the GP	hand-chosen points admitted for estimation only , never to steer the search